

AWS PYTHON WEB APPLICATION DEPLOYMENT USING DOCKER AND AWS EC2

PROJECT TITLE

AWS Python Web Application Deployment Using Docker and AWS EC2

Technologies Used:

Python, Flask, Docker, GitHub, AWS EC2, AWS ECR

AWS Region:

ap-south-1 (Mumbai)

Submitted By:

Krishna Kumar

Project Type:

Cloud / DevOps Deployment Project

Date:

September 2026

ABSTRACT

This project demonstrates the development and deployment of a simple Python web application using Flask. The application is containerized using Docker and deployed on an AWS EC2 instance.

The source code is maintained in a GitHub repository. Docker is used to package the application and its dependencies into a portable container. The Docker container is then executed on an Ubuntu-based AWS EC2 instance.

The project also demonstrates the use of AWS Elastic Container Registry (ECR) for storing Docker images and provides a basic understanding of cloud-based application deployment.

INTRODUCTION

Cloud computing allows applications to be deployed and accessed over the internet without depending on local systems.

In this project, a simple Python Flask web application is developed and containerized using Docker. The source code is stored in GitHub, while AWS EC2 is used as the cloud server for running the Docker container.

The project demonstrates the complete flow:

Developer → GitHub → Docker → AWS EC2 → Web Application

The application provides a web page and REST API endpoints that can be accessed through a web browser

OBJECTIVES

The main objectives of this project are:

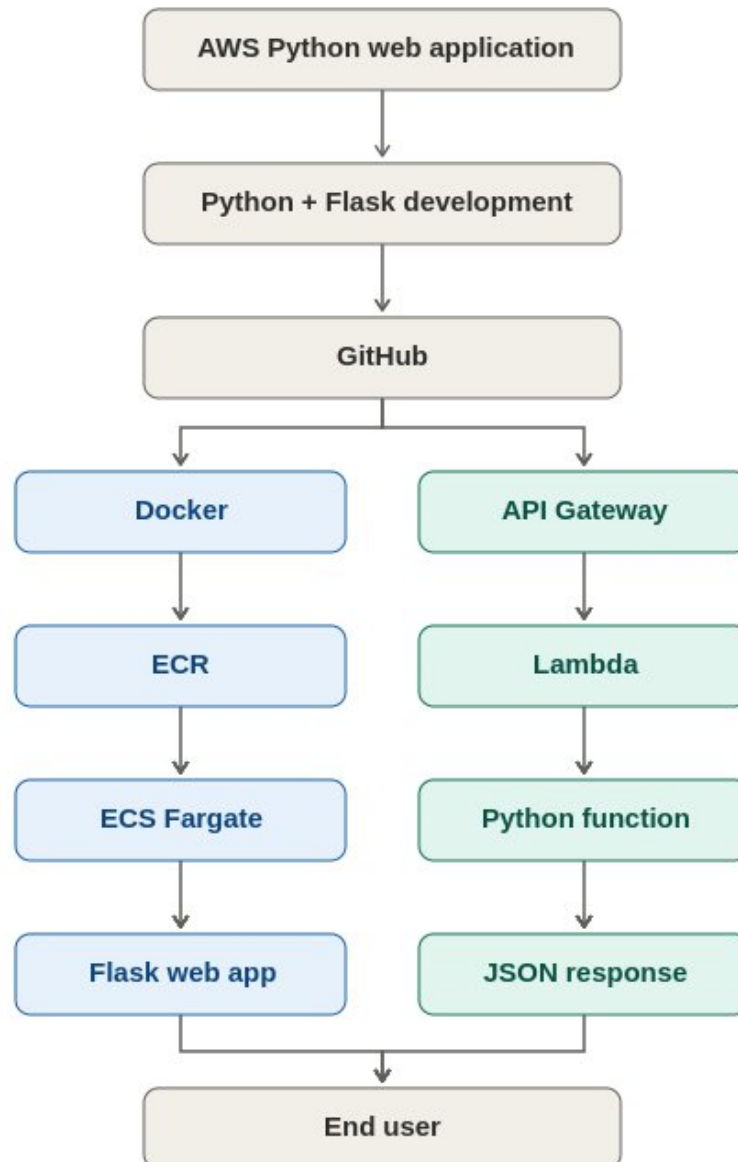
1. To develop a basic Python Flask web application.
2. To create a simple HTML frontend.
3. To create REST API endpoints using Flask.
4. To store the project source code in GitHub.
5. To create a Dockerfile for containerization.
6. To build a Docker image.
7. To run the application inside a Docker container.
8. To create and use an AWS EC2 instance.
9. To deploy the Docker container on EC2.
10. To understand AWS ECR for Docker image storage.
11. To access and test the application through the EC2 public IP address.

TECHNOLOGIES USED

Technology	Purpose
Python	Application development
Flask	Python web framework
HTML	Frontend development
JavaScript	API interaction
Docker	Application containerization
Git	Version control
GitHub	Source code repository
AWS EC2	Cloud server
AWS ECR	Docker image registry
Ubuntu	EC2 operating system
PowerShell	Local command-line environment

SYSTEM ARCHITECTURE

The system architecture shows how the Python Flask application moves from local development to cloud deployment. The application is first developed and tested locally. It is then containerized using Docker and stored in GitHub. Finally, the Dockerized application is deployed on an AWS EC2 Ubuntu server.



Architecture Explanation

The developer creates the Python Flask application and pushes the source code to GitHub.

The application is packaged into a Docker image using the Dockerfile. The Docker image can be stored in AWS ECR.

For deployment, the application is run inside a Docker container on an AWS EC2 Ubuntu server.

Users can access the application using:

`http://EC2_PUBLIC_IP:5000`

PROJECT STRUCTURE

The project contains the following files:

```
aws-python-webapp/  
├── app.py  
├── requirements.txt  
├── Dockerfile  
└── templates/  
    └── index.html
```

File Description

app.py

Contains the Flask application and API routes.

requirements.txt

Contains the Python dependencies required by the application.

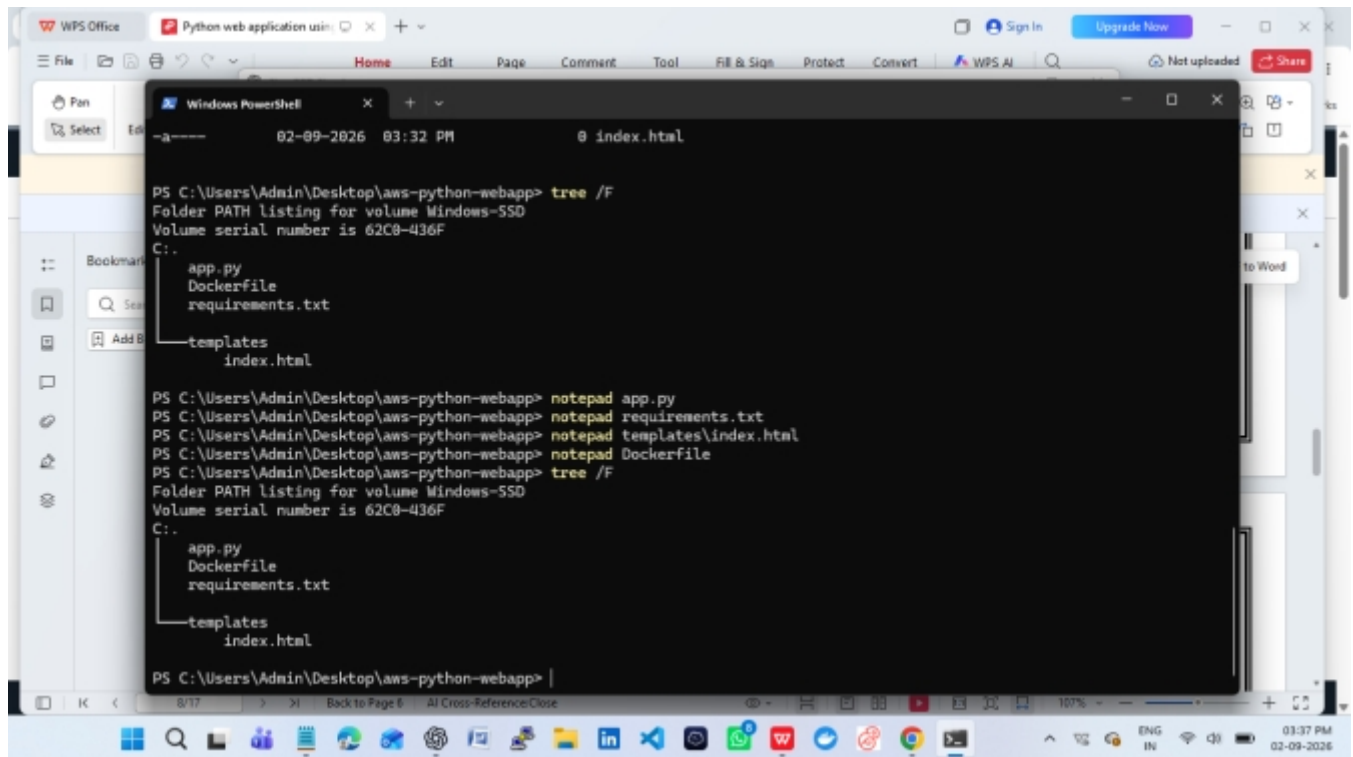
Dockerfile

Contains instructions for creating the Docker image.

templates/index.html

Contains the frontend HTML page

4. Project Folder Structure



The screenshot shows a Windows PowerShell terminal window with the following commands and output:

```
PS C:\Users\Admin\Desktop\aws-python-webapp> tree /F
Folder PATH listing for volume Windows-SSD
Volume serial number is 62C8-436F
C:.
├── app.py
├── Dockerfile
├── requirements.txt
└── templates
    └── index.html

PS C:\Users\Admin\Desktop\aws-python-webapp> notepad app.py
PS C:\Users\Admin\Desktop\aws-python-webapp> notepad requirements.txt
PS C:\Users\Admin\Desktop\aws-python-webapp> notepad templates\index.html
PS C:\Users\Admin\Desktop\aws-python-webapp> notepad Dockerfile
PS C:\Users\Admin\Desktop\aws-python-webapp> tree /F
Folder PATH listing for volume Windows-SSD
Volume serial number is 62C8-436F
C:.
├── app.py
├── Dockerfile
├── requirements.txt
└── templates
    └── index.html

PS C:\Users\Admin\Desktop\aws-python-webapp> |
```

The terminal window is titled "Windows PowerShell" and shows the current directory as "C:\Users\Admin\Desktop\aws-python-webapp". The output of the `tree /F` command shows the folder structure of the project, which includes a `templates` folder containing `index.html`, and three files: `app.py`, `Dockerfile`, and `requirements.txt`.

Create the Python Flask Application

Create the project folder and templates directory in PowerShell:

```
mkdir aws-python-webapp
cd aws-python-webapp
mkdir templates
```

Create app.py:

```
from flask import Flask, jsonify, render_template

app = Flask(__name__)

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/api/status")
def status():
    return jsonify({
        "message": "AWS Python Web App is running",
        "application": "Python Flask Web App",
        "status": "success"
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Create requirements.txt:

```
Flask==3.1.2
```

Create templates/index.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>AWS Python Web App</title>
</head>
<body>
  <h1>AWS Python Web App</h1>
  <p>Application is running successfully.</p>
  <p>Deployment: Python Flask</p>
</body>
</html>
```

Test the Application Locally

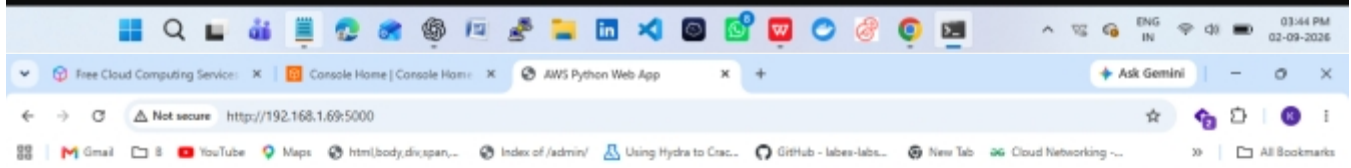
```
python -m venv venv  
venv\Scripts\activate  
pip install -r requirements.txt  
python app.py
```

Open the browser: <http://127.0.0.1:5000/>

API test: <http://127.0.0.1:5000/api/status>Expected API status: application = Python Flask Web App,
status = success.

```
Windows PowerShell
Collecting markupsafe>=2.1.1 (from Flask==3.1.2->r requirements.txt (line 1))
  Using cached markupsafe-3.0.3-py3-none-any.whl
Collecting werkzeug>=3.1.0 (from Flask==3.1.2->r requirements.txt (line 1))
  Using cached werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
  Using cached Flask-3.1.2-py3-none-any.whl (103 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Using cached click-8.5.0-py3-none-any.whl (125 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, Flask
Successfully installed Flask-3.1.2 blinker-1.9.0 click-8.5.0 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8

[notice] A new release of pip is available: 26.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS C:\Users\Admin\Desktop\aws-python-webapp> pip show Flask
Name: Flask
Version: 3.1.2
Summary: A simple framework for building complex web applications.
Home-page:
Author:
Author-email:
License-Expression: BSD-3-Clause
Location: C:\Users\Admin\Desktop\aws-python-webapp\venv\Lib\site-packages
Requires: blinker, click, itsdangerous, jinja2, markupsafe, werkzeug
Required-by:
(venv) PS C:\Users\Admin\Desktop\aws-python-webapp> python app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.1.69:5000
Press CTRL+C to quit
```



AWS Python Web Application

Running with Flask + Docker + AWS ECS

[Call API](#)

Hello from Python Flask!



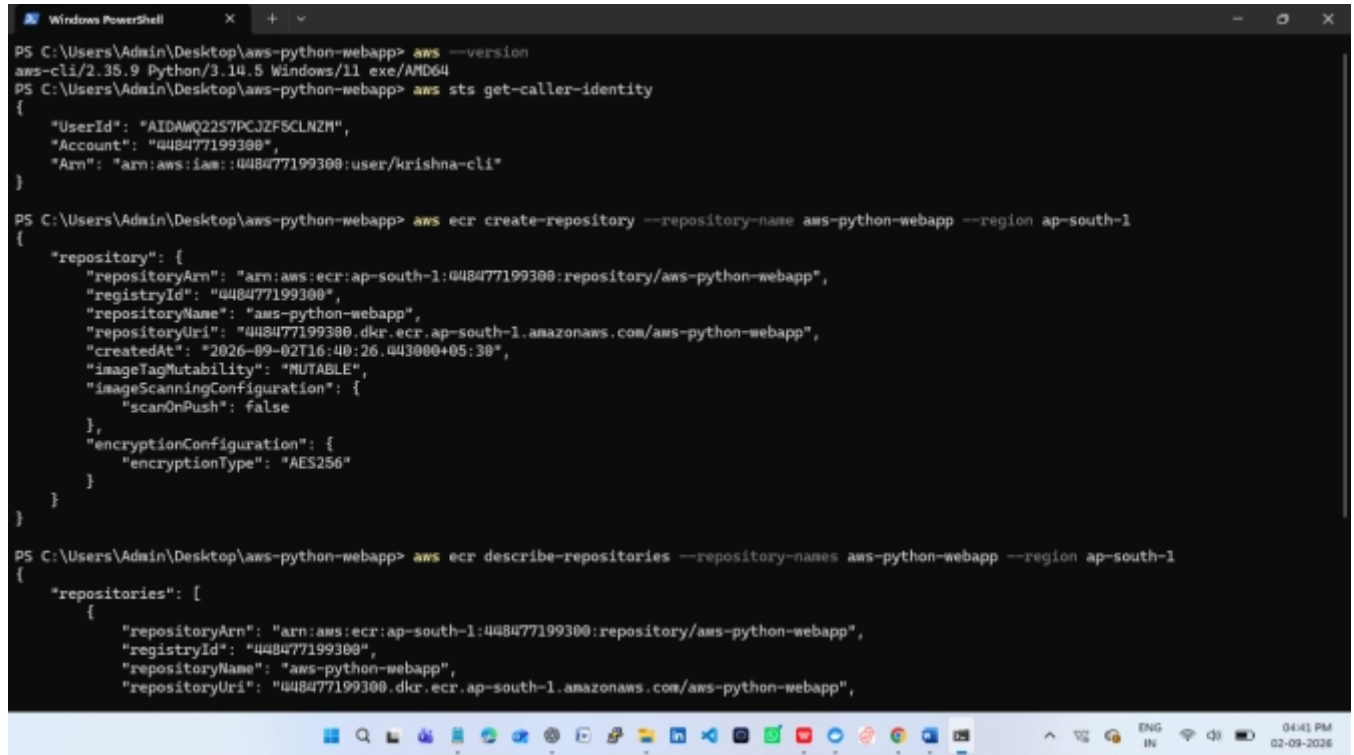
Configure AWS CLI

aws configure

aws sts get-caller-identity

Enter the AWS credentials, default region (ap-south-1), and output format (json) when prompted.

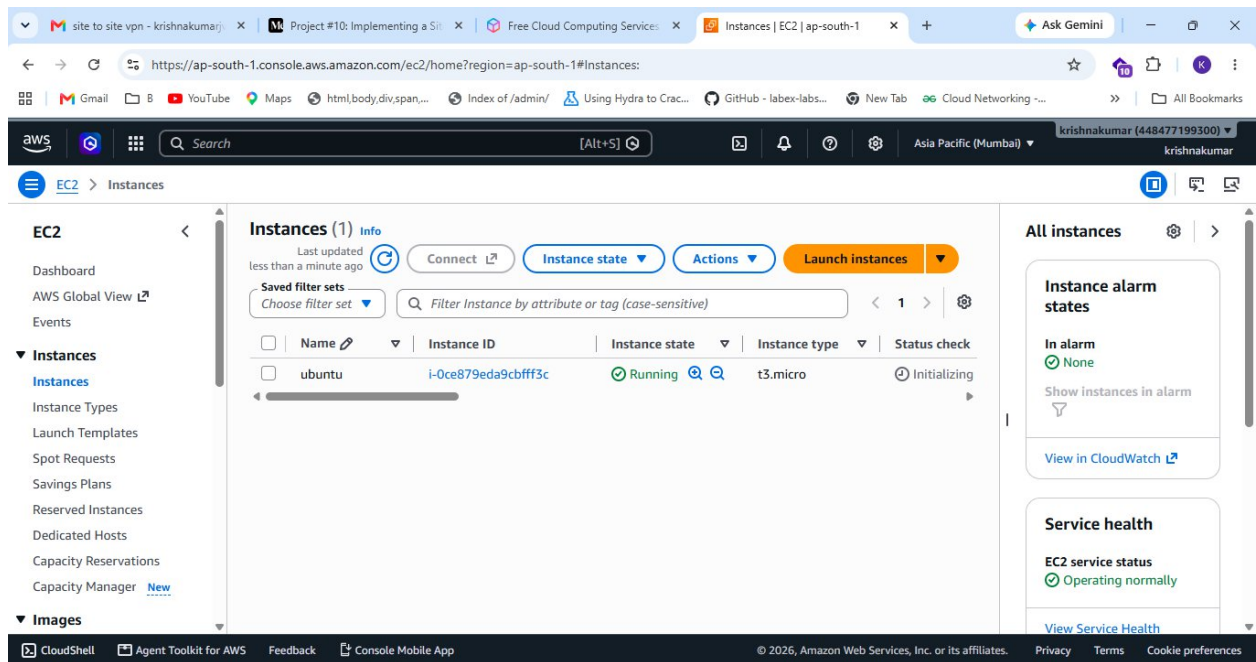
Security note: Never place AWS access keys, secret keys, passwords, or private keys in source code, Dockerfiles, GitHub, or screenshots.



```
PS C:\Users\Admin\Desktop\aws-python-webapp> aws --version
aws-cli/2.35.9 Python/3.14.5 Windows/x64 AMD64
PS C:\Users\Admin\Desktop\aws-python-webapp> aws sts get-caller-identity
{
  "UserId": "AIDAWQ22S7PCJZF5CLNZM",
  "Account": "448477199300",
  "Arn": "arn:aws:iam::448477199300:user/krishna-cli"
}
PS C:\Users\Admin\Desktop\aws-python-webapp> aws ecr create-repository --repository-name aws-python-webapp --region ap-south-1
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:ap-south-1:448477199300:repository/aws-python-webapp",
    "registryId": "448477199300",
    "repositoryName": "aws-python-webapp",
    "repositoryUri": "448477199300.dkr.ecr.ap-south-1.amazonaws.com/aws-python-webapp",
    "createdAt": "2026-09-02T16:40:26.043000+05:30",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
PS C:\Users\Admin\Desktop\aws-python-webapp> aws ecr describe-repositories --repository-names aws-python-webapp --region ap-south-1
{
  "repositories": [
    {
      "repositoryArn": "arn:aws:ecr:ap-south-1:448477199300:repository/aws-python-webapp",
      "registryId": "448477199300",
      "repositoryName": "aws-python-webapp",
      "repositoryUri": "448477199300.dkr.ecr.ap-south-1.amazonaws.com/aws-python-webapp",
```

Launch an Ubuntu EC2 Instance

1. Open AWS Console → EC2 → Instances → Launch instance.
2. Name: aws-python-webapp-ec2.
3. Choose an Ubuntu Server LTS AMI.
4. Choose a small eligible instance type suitable for your account/budget.
5. Create/select a key pair and download the .pem file.
6. Security Group: SSH TCP 22 from your IP only.
7. For direct Flask testing, allow TCP 5000 from your IP.
8. Launch the instance and copy its public IPv4 address.



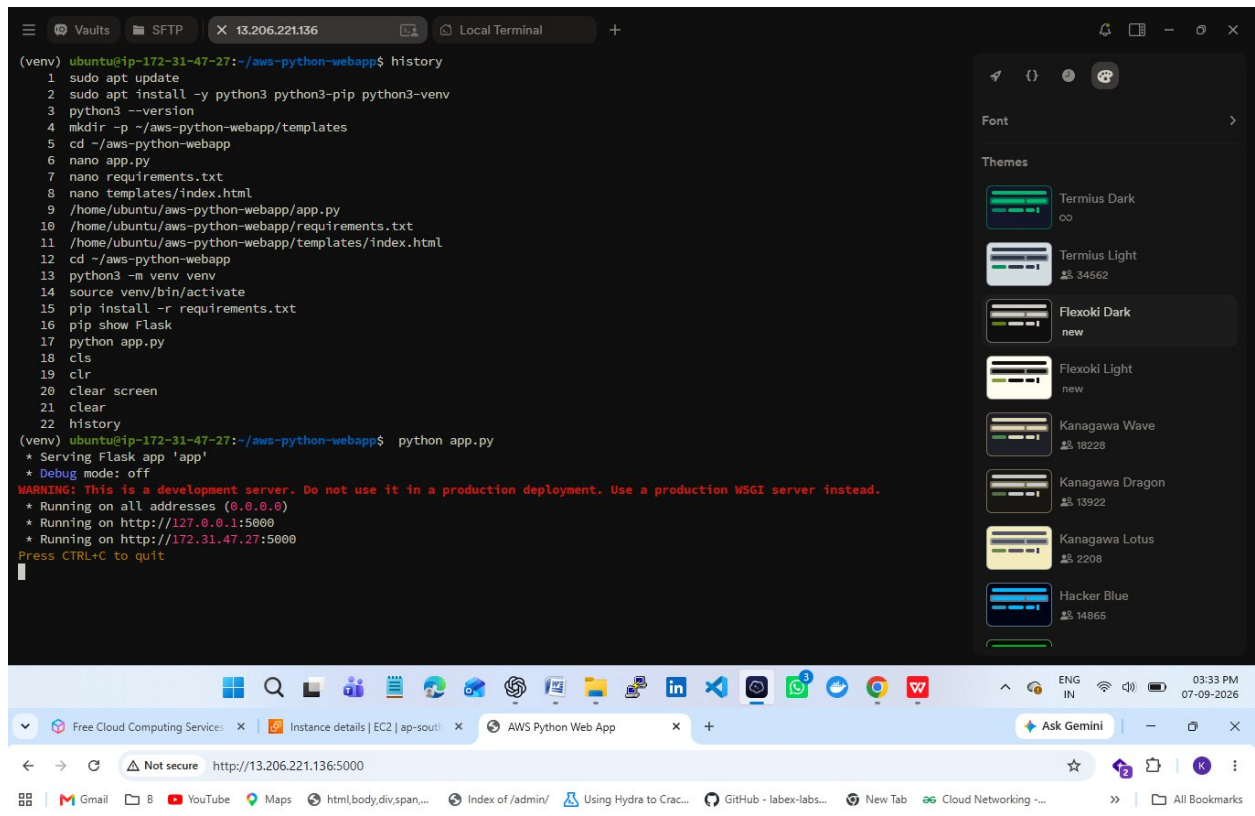
Install Python and Run the App on EC2

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip
mkdir -p ~/aws-python-webapp
cd ~/aws-python-webapp
```

Copy the project files to EC2 using Git, SCP, or another secure method. Then run:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python app.py
```

Open: http://<EC2_PUBLIC_IP>:5000/



Create the Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Save the file exactly as Dockerfile with no .txt extension.

Build and Test the Docker Image

```
docker build -t aws-python-webapp .
docker images
docker run -d --name aws-python-webapp-container -p 5000:5000 aws-python-webapp
docker ps
```

Open: <http://localhost:5000/>

```
docker stop aws-python-webapp-container
docker rm aws-python-webapp-container
```

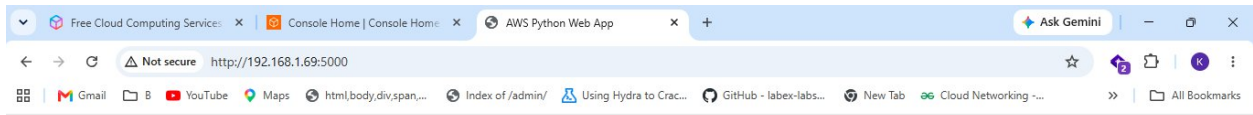


```
Windows PowerShell
(venv) PS C:\Users\Admin\Desktop\aws-python-webapp> docker build -t aws-python-webapp .
[+] Building 11.2s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 212B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea
=> => resolve docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea
=> [internal] load build context
=> => transferring context: 17.51MB
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:8d773a3f6ee6fcf2cb3b9b17e8c3542b9656875b6ea9d5960994784f77075a3a
=> => exporting config sha256:6d36c555eb579ed9b88290eb1e363c67b3ccc4ce31c92e3a1e156b0006bf2a40
=> => exporting attestation manifest sha256:24fb3d7a5abc3a180c7872369ace3f105c0232e9542ee3dd4864e665940e184b
=> => exporting manifest list sha256:3c54922a7bfe190756a12a97d5b2413d700b74bfb1201855e0c2f8776d9e621b
=> => naming to docker.io/library/aws-python-webapp:latest
=> => unpacking to docker.io/library/aws-python-webapp:latest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/1zc8win23240tc7b3ahhfrcdx
(venv) PS C:\Users\Admin\Desktop\aws-python-webapp> docker images

INFO In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
aws-python-webapp:latest 3c54922a7bfe 236MB 57.4MB
(venv) PS C:\Users\Admin\Desktop\aws-python-webapp>

Windows PowerShell
(venv) PS C:\Users\Admin\Desktop\aws-python-webapp> docker run -d -p 5000:5000 --name python-webapp aws-python-webapp
3ec3e46638192d68a062e552acafa03c4b94a54f8f25dbb44d784f69e9e6314c
(venv) PS C:\Users\Admin\Desktop\aws-python-webapp> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
3ec3e4663819 aws-python-webapp "python app.py" 19 seconds ago Up 17 seconds 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp python-webapp
(venv) PS C:\Users\Admin\Desktop\aws-python-webapp>
```



AWS Python Web Application

Running with Flask + Docker + AWS ECS

[Call API](#)

Hello from Python Flask!



Create Amazon ECR Repository

AWS Console → ECR → Private repositories → Create repository.

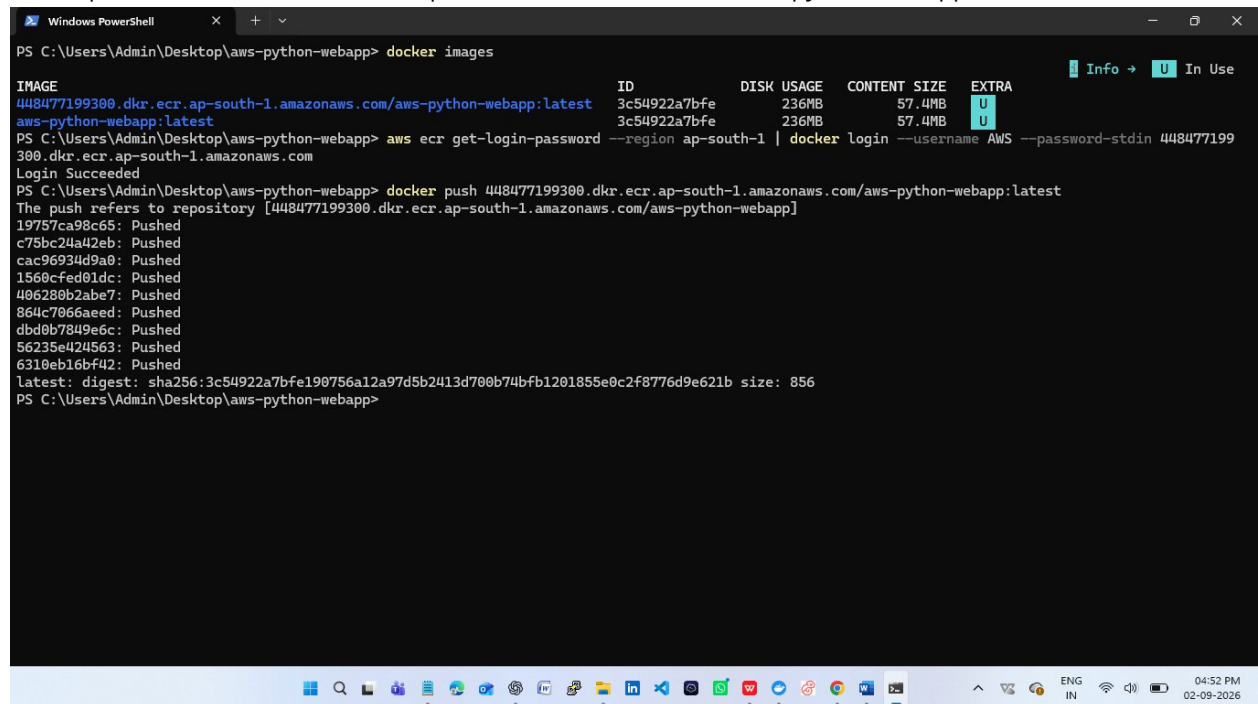
Repository name: aws-python-webapp

Push Docker Image to ECR

```
aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin  
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com
```

```
docker tag aws-python-webapp:latest <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/aws-python-  
webapp:latest
```

```
docker push <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/aws-python-webapp:latest
```



```
Windows PowerShell
PS C:\Users\Admin\Desktop\aws-python-webapp> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
448477199300.dkr.ecr.ap-south-1.amazonaws.com/aws-python-webapp:latest	3c54922a7bfe	236MB	57.4MB	U
aws-python-webapp:latest	3c54922a7bfe	236MB	57.4MB	U

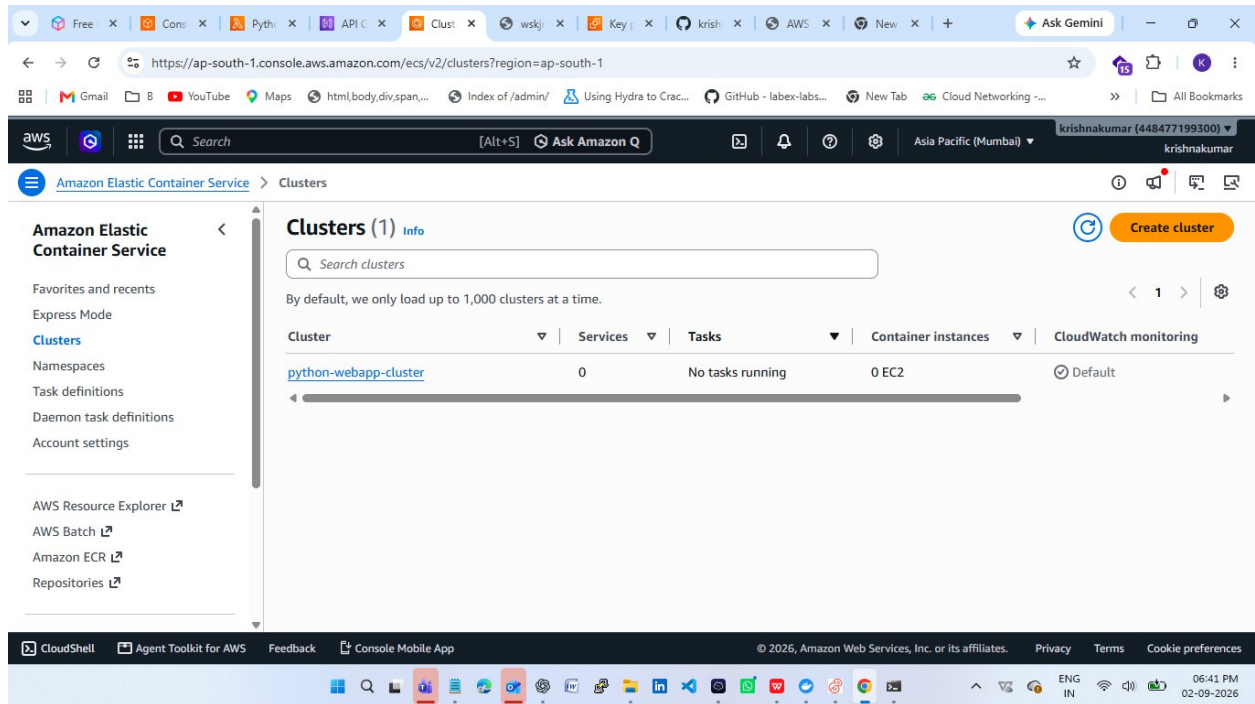
```
PS C:\Users\Admin\Desktop\aws-python-webapp> aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 448477199300.dkr.ecr.ap-south-1.amazonaws.com
Login Succeeded
PS C:\Users\Admin\Desktop\aws-python-webapp> docker push 448477199300.dkr.ecr.ap-south-1.amazonaws.com/aws-python-webapp:latest
The push refers to repository [448477199300.dkr.ecr.ap-south-1.amazonaws.com/aws-python-webapp]
19757ca98c65: Pushed
c75bc24a42eb: Pushed
cac96934d9a0: Pushed
1560cfed01dc: Pushed
406280b2abe7: Pushed
864c7066aeed: Pushed
dbd0b7849e6c: Pushed
56235e44563: Pushed
6310eb16bf42: Pushed
latest: digest: sha256:3c54922a7bfe190756a12a97d5b2413d700b74bfb1201855e0c2f8776d9e621b size: 856
PS C:\Users\Admin\Desktop\aws-python-webapp>
```

Create ECS Cluster

AWS Console → ECS → Clusters → Create cluster.

Cluster name: aws-python-webapp-cluster

Use ECS with AWS Fargate so the container can run without managing an ECS EC2 worker fleet.



Create ECS Task Definition

- Family: aws-python-webapp-task.
- Compatibility: Fargate.
- CPU: 0.25 vCPU / 256 CPU units.
- Memory: 0.5 GB / 512 MiB.
- Task execution role: ECS task execution role with AmazonECSTaskExecutionRolePolicy.
- Container name: aws-python-webapp.
- Image URI: your ECR image URI ending in :latest.
- Container port: 5000 TCP.

The screenshot displays the AWS Management Console for the Amazon Elastic Container Service (ECS). The breadcrumb navigation shows the path: Amazon Elastic Container Service > Task definitions > python-webapp-task > Revision 1 > Containers. The main content area is titled 'python-webapp-task:1' and shows it was last updated on September 2, 2026, at 18:40 (UTC+5:30). There are buttons for 'Deploy', 'Actions', and 'Create new revision'. The 'Overview' tab is selected, displaying a grid of key information:

ARN arn:aws:ecs:ap-south-1:448477199300:task-definition/python-webapp-task:1	Status ACTIVE	Time created September 2, 2026, 18:40 (UTC+5:30)	App environment Fargate
Task role -	Task execution role ecsTaskExecutionRole	Operating system/Architecture Linux/X86_64	Network mode awsvpc
Fault injection Turned off			

Below the overview, there are tabs for 'Containers', 'JSON', 'Task placement', 'Volumes (0)', 'Requires attributes', and 'Tags'. The 'Containers' tab is active, showing the 'Task size' section.

The bottom of the screenshot shows the Windows taskbar with various application icons and the system clock indicating 06:41 PM on 02-09-2026.

Create ECS Service

1. Open ECS → Clusters → aws-python-webapp-cluster → Create service.
2. Compute option: Fargate.
3. Task definition: aws-python-webapp-task.
4. Service name: aws-python-webapp-service.
5. Desired tasks: 1.
6. Select the VPC and public subnets.
7. Allow TCP 5000 from your IP for learning/testing, or from a load balancer security group if an ALB is used.
8. Enable public IP for direct testing without a load balancer.
9. Create the service.

Free x Cons x Pyth x API C x Clust x wskj x Key x krish x AWS x New x + Ask Gemini

https://ap-south-1.console.aws.amazon.com/ecs/v2/clusters/python-webapp-cluster/services?region=ap-south-1

aws Search [Alt+S] Asia Pacific (Mumbai) krishnakumar (448477199300) krishnakumar

Amazon Elastic Container Service > Clusters > python-webapp-cluster > Services

python-webapp-cluster Last updated September 2, 2026, 18:46 (UTC+5:30) Actions Create with Express Mode

Cluster overview

Cluster ARN arn:aws:ecs:ap-south-1:44...	Status Active	CloudWatch monitoring Default	Action logs Turned off
Services 1 active 0 draining	Tasks 1 running 0 pending	Registered container instances -	

< Services Tasks Daemons Infrastructure Metrics Scheduled tasks Configuration - updated >

Services (1) Info Last updated September 2, 2026, 18:46 (UTC+5:30) Manage tags Update Delete service Create

Filter services by value Filter launch type Any launch type Filter scheduling strategy Any scheduling strategy Filter resource management type Any resource management type

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Free Cl Consol Pythor API Gat Task co wskjniir Key pai krishna AWS Py New To AW x + Ask Gemini

Not secure http://65.2.74.101:5000

Gmail B YouTube Maps html,body,div,span,... Index of /admin/ Using Hydra to Crac... GitHub - labex-labs... New Tab Cloud Networking ... All Bookmarks

AWS Python Web Application

Running with Flask + Docker + AWS ECS

Call API

Free Cl Consol Pythor API Gat Task co wskjniir Key pai krishna AWS Py New To AW x + Ask Gemini

Not secure http://65.2.74.101:5000

Gmail B YouTube Maps html,body,div,span,... Index of /admin/ Using Hydra to Crac... GitHub - labex-labs... New Tab Cloud Networking ... All Bookmarks

06:51 PM 02-09-2026

Lambda Function (Serverless Demonstration)

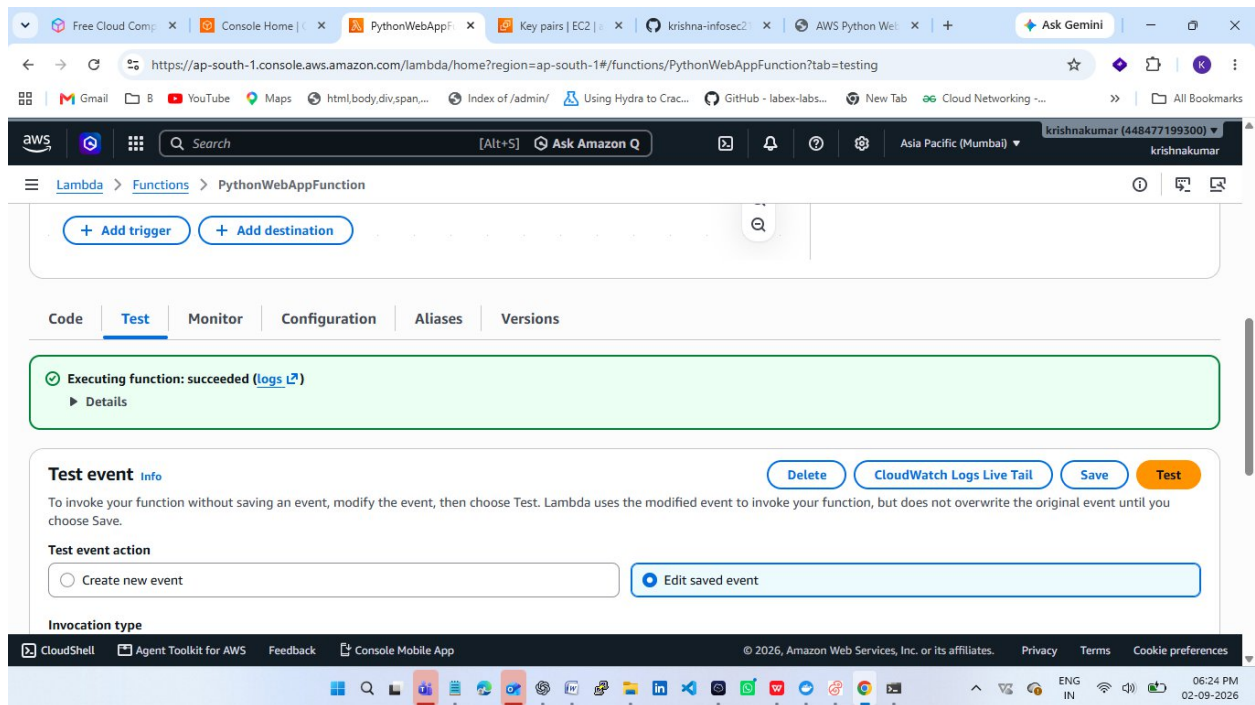
AWS Console → Lambda → Create function → Author from scratch.

Function name: aws-python-webapp-lambda. Choose a currently supported Python runtime.

```
import json

def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps({
            "message": "Hello from AWS Lambda!",
            "application": "Python Web App",
            "status": "success"
        })
    }
```

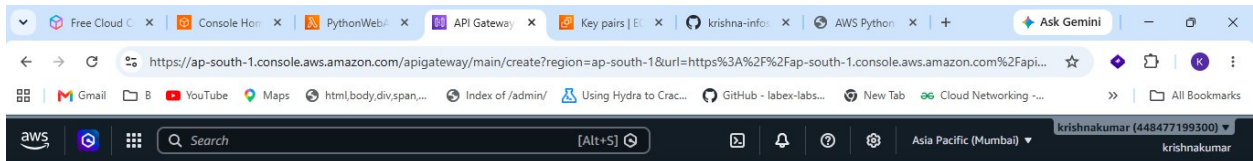
Choose Deploy and test the function using the Lambda Test tab.



API Gateway HTTP API

1. Open API Gateway → APIs → Create API.
2. Under HTTP API, choose Build.
3. Add Lambda integration and select aws-python-webapp-lambda.
4. API name: aws-python-webapp-api.
5. Route: GET /hello.
6. Create the API and copy the Invoke URL.

Test: <API_INVOKE_URL>/hello



Step 1

Configure API

Step 2 - optional

Configure routes

Step 3 - optional

Define stages

Step 4

Review and create

Configure routes - optional

Configure routes Info

API Gateway uses routes to expose integrations to consumers of your API. Routes for HTTP APIs consist of two parts: an HTTP method and a resource path (e.g., GET /pets). You can define specific HTTP methods for your integration (GET, POST, PUT, PATCH, HEAD, OPTIONS, and DELETE) or use the ANY method to match all methods that you haven't defined on a given resource.

Method

Resource path

Integration target

GET

/hello

PythonWebAppFunction

Remove

Add route

Cancel

Review and create

Previous

Next



API Gateway

APIs

Custom domain names

Domain name access associations

VPC links

API: pythonwebappfun... (wskjniir5h)

Develop

Routes

Authorization

Integrations

CORS

Reimport

pythonwebappfunction

Stage: - Deploy

Edit

API details

API ID

wskjniir5h

Protocol

HTTP

Created

2026-09-02

Description

-

IP address type

IPv4

Default endpoint

Enabled

https://wskjniir5h.execute-api.ap-south-1.amazonaws.com

ARN

arn:aws:apigateway:ap-south-1::apis/wskjniir5h

Stages for pythonwebappfunction (1)

Find resources

Free Cloud x Console x PythonW x API Gate x wskjniir5i x Key pairs x krishna x AWS Pyti x + Ask Gemini x

https://wskjniir5h.execute-api-south-1.amazonaws.com/hello

Gmail 8 YouTube Maps html,body,div,span,... Index of /admin/ Using Hydra to Crac... GitHub - labex-labs... New Tab Cloud Networking ... All Bookmarks

Pretty-print

```
{
  "message": "Hello from AWS Lambda!",
  "application": "Python Web App",
  "status": "success"
}
```

Windows taskbar with icons for File Explorer, Edge, Word, Teams, Outlook, OneDrive, and various social media apps. System tray shows language (ENG IN), network, and time (06:32 PM 02-09-2026).

GitHub Repository Preparation

Project folder:

aws-python-webapp/

```
|— app.py
|— requirements.txt
|— Dockerfile
|— templates/
|   |— index.html
|— README.md
```

Initialize Git

```
git init
git add .
git commit -m "Initial AWS Python web application project"
```

Create GitHub Repository

GitHub → **New Repository** → Repository name: [aws-python-webapp](#)

Create repository.

Then EC2 terminal:

```
git branch -M main  
git remote add origin https://github.com/YOUR_USERNAME/aws-python-webapp.git
```

git push -u origin main

The screenshot shows a Windows PowerShell terminal window and a web browser displaying a GitHub repository.

PowerShell Terminal:

```
Directory: C:\Users\Admin\Desktop\aws-python-webapp

Mode                LastWriteTime         Length Name
----                -
d-----            02-09-2026   03:32 PM             templates
d-----            02-09-2026   03:38 PM             venv
-a-----            02-09-2026   03:35 PM             551 app.py
-a-----            02-09-2026   03:37 PM             173 Dockerfile
-a-----            02-09-2026   03:35 PM             12 requirements.txt

PS C:\Users\Admin\Desktop\aws-python-webapp> git init
Initialized empty Git repository in C:/Users/Admin/Desktop/aws-python-webapp/.git/
PS C:\Users\Admin\Desktop\aws-python-webapp> git add .
PS C:\Users\Admin\Desktop\aws-python-webapp> git commit -m "Initial Python Flask web application"
[master (root-commit) 4b3fac6] Initial Python Flask web application
 4 files changed, 70 insertions(+)
 create mode 100644 Dockerfile
 create mode 100644 app.py
 create mode 100644 requirements.txt
 create mode 100644 templates/index.html
PS C:\Users\Admin\Desktop\aws-python-webapp> git branch -M main
PS C:\Users\Admin\Desktop\aws-python-webapp> git remote add origin https://github.com/krishna-infosec21/aws-python-webapp.git
PS C:\Users\Admin\Desktop\aws-python-webapp> git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 1.12 KiB | 143.00 KiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/krishna-infosec21/aws-python-webapp.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Admin\Desktop\aws-python-webapp>
```

Web Browser:

The browser shows the GitHub repository page for `krishna-infosec21 / aws-python-webapp`. The repository is public and has 0 stars, 0 forks, and 0 watchers. The commit history shows a single commit by `krishna-infosec21` titled "Initial Python Flask web application" with commit hash `4b3fac6`. The repository contains the following files:

- `templates`
- `Dockerfile`
- `app.py`
- `requirements.txt`

The README section is currently blank.

Testing Checklist

- ☐ Flask application works locally.
- ☐ Local /api/status returns success.
- ☐ EC2 deployment works.
- ☐ Docker image builds successfully.
- ☐ Docker container runs successfully.
- ☐ Docker image is available in ECR.
- ☐ ECS/Fargate service has one running task.
- ☐ ECS application response works.
- ☐ Lambda test returns HTTP 200.
- ☐ API Gateway GET /hello works.
- ☐ .gitignore is present.
- ☐ No AWS credentials, private keys, or secrets are in GitHub.

Project Outcome

The completed project demonstrates practical cloud deployment skills using Python Flask, Amazon EC2, Docker, Amazon ECR, Amazon ECS/Fargate, AWS Lambda, API Gateway, IAM and networking. The GitHub repository provides a reproducible source-code record, while this Word report provides the deployment procedure and screenshot evidence.

